Agent = Harness(Model): Five Mechanistic Primitives, and What Is Not One

Chase Dovey
Average Joes Lab
https://cdovey.dev

***Abstract*—The software around a language model in an agent is called a scaffold, a framework, a runtime or a harness, and none of these names says what it contains. We take the position that an LLM agent is a model wrapped in a harness, *Agent* = *Harness*(*Model*), and that every harness is built from five mechanistic primitives: INPUT, CONTEXT, MODEL INTERFACE, OUTPUT and CONTROL FLOW. Each is defined by what it does and by what it never does, so the five are mutually exclusive. Everything else said about a harness is at one of two other levels: a *design decision* within a primitive, which is where harnesses differ, or a *production concern* (persistence, safety and security, observability, resilience, performance, evaluation), a question asked of every primitive rather than a part of the harness. As evidence, we deconstruct six harnesses from their primary sources at pinned versions: Claude Code, Codex CLI, opencode, pi, Hermes Agent and a LangGraph workflow. Every component found has a home in one primitive or one concern, and the differences between the six are design decisions. The vocabulary gives a procedure for locating a failure in one of five places before attributing it to the model, and a schema for reporting the harness alongside the model. The evidence for every assignment, and the minimal harnesses the definitions were drawn from, are published with the paper.**

***Index Terms*—LLM agents, agent harness, scaffold, agent architecture, software engineering agents**

I. INTRODUCTION

A language model does one thing: given a sequence of tokens, it produces a distribution over the next token. It cannot see a file it was not shown, remember a conversation it was not sent again, or carry out a command it wrote. Language models are nonetheless described as agents that read codebases, run tests and fix bugs. The difference between what a model is and what an agent does is software around the model, and that software is the subject of this paper.

The software has many names. Evaluations call it a *scaffold* [1], [2]; libraries call themselves *frameworks*, *SDKs* or an *orchestration framework and runtime* [3]; and *harness* is now common among people who build coding agents [4]. None of the names says what the thing contains, where it ends and the model begins, or how two instances of it differ. Agents are therefore compared by the model inside them, although results vary with the system around it (§VI), and when an agent fails there is no agreed list of places to look.

a) Position: We argue three claims.

1) **An LLM agent is a model wrapped in a harness**: *Agent* = *Harness*(*Model*). The model maps tokens to a next token. The harness is everything else on the path of a request.
2) **Every harness is built from five mechanistic primitives**: INPUT, CONTEXT, MODEL INTERFACE, OUTPUT and CONTROL FLOW. Each is defined by what it does and by what it never does, so that each piece of a harness has one home.
3) **Everything else said about a harness is at one of two other levels**: a *design decision* within a primitive (one tool or forty; a loop or a graph), or a *production concern* asked of every primitive (persistence, safety and security, observability, resilience, performance, evaluation). Neither adds a primitive.

The vocabulary separates what a harness *is* from what a given harness *chose* and from what makes it fit to *run*, so that a failure can be located in one of five places before it is attributed to the model, and a benchmark result can report the harness that produced it.

b) Evidence: The definitions come from building a harness from nothing, one primitive at a time, around a fixed model and task, and observing what each version could not do; the five programs are published with the paper [5]. The claim that the five cover existing harnesses is tested by deconstructing six of them from their primary sources at pinned versions (§V): every component on the request path is assigned to one primitive, or to one concern together with the primitive it changes, and every case that was hard to place is recorded. The evidence, a cited assignment for each of 548 components, is published with the paper. The full paper, with the per-harness deconstructions, the rules in full and the appendices, is available as a preprint; this paper is its summary, shared non-archivally. This is a conceptual contribution: it reports no experiment in which a primitive is varied, and it does not prove that no sixth primitive could ever be needed. It states what would refute the claim (§VII) and gives a deconstruction a reader can check.

II. THE BOUNDARY: MODEL AND HARNESS

a) The model: We take a language model to be a function from a token sequence to a distribution over the next token, applied autoregressively until a stop token is produced: *TokensOut* = *Model*(*TokensIn*). Chat templates, tool-call formats and reasoning tokens are conventions for laying out *TokensIn* and reading *TokensOut*. Models learn these conventions in training, and can learn to emit calls to tools [6], but the calls are carried out, and the conventions written and parsed, by something else. Viewed from outside, the model

Algorithm 1 A harness. Lines are labelled with the primitive they implement.

```
1:  procedure ControlFlow( )
2:      conv ← [ ]
3:      loop                                  ▷ outer loop: one request at a time
4:          conv.append(User(Input( )))                    ▷ Input: a person
5:          loop                  ▷ inner loop: one request, carried through
6:              ctx ← Context(conv)                              ▷ Context
7:              resp ← ModelInterface(ctx)                ▷ Model interface
8:              conv.append(Assistant(resp))
9:              results ← Output(resp)                ▷ Output: words, actions
10:             if results = ∅ then break              ▷ nothing ran: hand back
11:             conv.append(User(results))                     ▷ Input: the world
```

keeps no state between calls, cannot read from or act on its environment, and cannot call itself again.

b) The harness: The harness is everything in an agent that is not the model: the software that turns the world into tokens for the model, and the model's tokens into effects on the world. The boundary is at the model's weights and its sampling. Everything the model is shown, everything it can do, and everything that happens between one call and the next is on the harness side, including things often attributed to the model: memory across sessions, tool use, and knowledge of the date or the working directory. Algorithm 1 gives the shape every harness shares, each line labelled with its primitive: something comes in, the harness assembles a context around it and sends it to the model, and the harness handles the response, showing words to a person or carrying out the actions requested. If an action produced a result, the result comes back in and the loop runs again; if not, control returns to the person.

c) Scope: The primitives describe the path of a request at run time; hosting, packaging, configuration loading and the user-interface framework are out of scope, and the model is held fixed. What makes a harness an agent's is a CONTROL FLOW that feeds the results of actions back to the model.

III. The Five Primitives

For each primitive we give its *mechanism*, its *negative definition* (what it never does, and which primitive does), and its *design space*. The negative definitions make the five mutually exclusive: each piece of a harness has one home, assigned by its function rather than by its name or its location in the code.

A. Input: *capture what comes in*

Mechanism. Capture what enters the agent and hand it to CONTEXT. There are two sources: a person (a message, a file, an interrupt, an answer to a question the model asked) and the world (the result of an action, an event, a scheduled trigger). **Never.** INPUT never decides what the model is shown; that is CONTEXT. It never runs anything; that is OUTPUT. An answer a person gives that the *harness* consumes, such as an approval, is a CONTROL FLOW decision; an answer the *model* consumes, returned as a tool result, is input. **Design space.** The source (terminal, chat channel, webhook) and the kind (text, files, results, interrupts).

B. Context: *assemble what the model is shown*

Mechanism. Assemble, for one call, everything the model is shown: instructions, the tools' descriptions, the conversation so far, and anything retrieved from memory or the environment; and fit it to the window by truncation, compaction or selection. CONTEXT is the agent's working memory, rebuilt on every call. **Never.** CONTEXT never calls the model; that is MODEL INTERFACE. It never acts on the world; that is OUTPUT. Reading a memory store is CONTEXT; writing one, if the model requests it, is an action and so OUTPUT. Code that decides what the model sees is CONTEXT even when it sits inside a tool. **Design space.** What the instructions hold; how tools are described; how long-term memory comes in (preloaded, retrieved, or searched by the agent); the roles each piece is placed in; what happens when the window fills.

C. Model interface: *reach the model*

Mechanism. Send the assembled context to the model and bring back its response in a form the rest of the harness can read: words, and structured requests to act. This includes the client, the wire format, the translation to the model's chat template, streaming, retries, and switching to a backup model when a request could not be served. **Never.** MODEL INTERFACE never decides what the request contains; that is CONTEXT. It never acts on the response; that is OUTPUT. It receives a stream; rendering each fragment is OUTPUT. A change of model because the request could not be served is MODEL INTERFACE; a change of model keyed to the work (planning, a cheap model for a side task) is CONTROL FLOW. **Design space.** A hosted API or local inference; one provider or many; native tool-call formats or calls parsed from text; retry policy and backup models. MODEL INTERFACE is the only primitive that knows which model is inside, which is why changing the model means changing it alone.

D. Output: *carry the response out*

Mechanism. Handle what the model returned: show its words to a person, and carry out each requested action, collecting its result. The tool set, the executor, and the environment an action runs in are OUTPUT. **Never.** OUTPUT never decides whether to go again; that is CONTROL FLOW. It never assembles the next request; that is CONTEXT. The model only ever *requests* an action; OUTPUT decides whether and how it happens, which is where sandboxes and never-lists live. Several tool calls from one response run concurrently: OUTPUT. Several *model* calls run concurrently: CONTROL FLOW. **Design space.** The tool set, from one shell to dozens of specific tools; the action format, from structured calls to model-written code [7]; the execution environment; and the presentation of results, which [8] show affects performance.

E. Control flow: *decide what happens next*

Mechanism. Define the shape of the loop: send results back and go again, hand back to the person, or stop. Step and token budgets, approval gates the harness consumes, delegation to sub-agents, and routing to a model keyed to the work are all

CONTROL FLOW. **Never.** CONTROL FLOW never captures input, assembles context, calls the model or runs a tool. It sequences the other four. A workflow in which code fixes the order of model calls [9] and an agent in which the model directs its own process differ here and only here. **Design space.** An open loop, a fixed pipeline or an explicit graph; sub-agents; approval modes; stopping rules.

a) Necessity: Each primitive answers a question no other answers. The five minimal harnesses published with the paper were built one primitive at a time around a fixed model and task, and each version fails in a specific way without the next: without OUTPUT the model cannot act, without CONTROL FLOW a result goes nowhere, without CONTEXT the model spends actions finding out where it is. This shows that each primitive removes a failure the others do not, not that no sixth could be needed; §V tests that.

IV. THREE LEVELS: PRIMITIVE, DESIGN DECISION, PRODUCTION CONCERN

Descriptions of harnesses mix three kinds of statement. "It has a loop" names a primitive. "It gives the model forty tools" names a decision within one. "It runs commands in a sandbox" names a measure taken for a reason that applies to every primitive. Keeping the three apart is the main use of the vocabulary.

a) Primitives: A primitive is a mechanism every harness has, defined by its function on the request path. A harness has exactly five.

b) Design decisions: A design decision is one way of implementing a primitive. It is the level at which harnesses differ and at which they are usually described: a single shell tool or a set of file tools (OUTPUT); a loop or a graph (CONTROL FLOW); one provider or a gateway to many (MODEL INTERFACE). A design decision never adds a primitive; it fills one. The test is whether a different harness could choose differently and still be a harness.

c) Production concerns: A production concern is a question asked of every primitive, whose answer changes how some primitives are implemented. We use six (Table I): what should outlast the session (persistence); what could do harm, and who could make it (safety and security); what did it do (observability); what if something breaks (resilience); what is slow or costly (performance); and how do we know it still works (evaluation). A concern is not a part of the harness. A sandbox is not a sixth primitive; it is the safety answer in OUTPUT. A memory store is not a sixth primitive; it is the persistence answer in CONTEXT and OUTPUT. The test is whether the thing is motivated by a question that could equally be asked of a different primitive.

d) Rules for the cases at the boundaries: The negative definitions decide most cases. Thirteen rules, each needed for a component found in §V, decide the rest; those used below: a component is assigned by its *function, not its location*, so an approval decision inside a tool's runtime is CONTROL FLOW; a *program the model writes* at run time is CONTROL FLOW if it sequences model calls and OUTPUT if it sequences only tools; a *sub-agent* is a harness of its own, and delegation to it is CONTROL FLOW of the outer harness; and one primitive may use another *as a service*, as when CONTEXT asks the model to summarize old turns before compacting them, in which case the component is assigned by what it *decides* and the call it makes is a use of MODEL INTERFACE. The negative definitions are about responsibility, not about which function calls which.

V. DECONSTRUCTION OF SIX HARNESSES

If the position holds, a harness built by someone else, for a different purpose, at a different scale, should decompose into the five primitives with nothing left over, and its differences from other harnesses should be locatable as design decisions within a primitive. We test this on six.

a) Selection: We chose harnesses in wide use whose primary sources were available, across three kinds: two command-line coding agents from model developers (Claude Code [10] and Codex CLI [11]), two open-source coding agents built outside the model developers (opencode [12] and pi [13]), one general-purpose agent built for messaging platforms as well as the terminal (Hermes Agent [14]), and one workflow framework (LangGraph [3]), for which we deconstruct the canonical ReAct agent it ships together with the runtime that executes it. Five of the six publish their source; Claude Code does not, and is deconstructed from its documentation.

b) Method: For each harness we (i) read the source at a pinned commit, or the documentation on a stated date, and mapped the path of a request from the moment a message arrives to the next model call; (ii) assigned every component on that path to exactly one primitive, or to one production concern together with the primitive it changes, by the definitions of §III and the rules of §IV-0d, citing a file and line range or a page and section for each; and (iii) recorded every component that was hard to place. The assignments were made with an AI coding agent as a research instrument, under the author's direction and review, and each can be checked against its cited source without the agent.

c) Summary: Table II gives the six side by side, one row per primitive. Every component we found has a home, and the differences between the harnesses are in the rows. None of the six required a sixth row.

d) What was hard to place: Three cases recur across harnesses.

- **Primitives that call the model.** In Claude Code, opencode and pi, compaction decides what to summarize and then calls the model to summarize it; Claude Code's web-fetch tool and pi's code mode call models from inside a tool. Read as a call graph, these break the rule that CONTEXT and OUTPUT never call the model; read as responsibility, they do not, since the component still decides one thing and uses MODEL INTERFACE to do it. The rule on primitives as services was added for them.
- **Approval gates inside tools.** In opencode and Hermes the code that asks the person sits inside the tool that would act, and in Codex the approval requirement depends on what the sandbox allows. By function the gate is CONTROL FLOW

TABLE I

SIX PRODUCTION CONCERNS AS QUESTIONS ASKED OF EACH PRIMITIVE. EACH CELL NAMES WHAT THE CONCERN CHANGES IN THAT PRIMITIVE; "—" MEANS NOTHING ESSENTIAL WAS FOUND THERE IN THE HARNESSES EXAMINED.

	Persistence *What outlasts the session?*	**Safety & security** *What could do harm, and who could make it?*	**Observability** *What did it do?*	**Resilience** *What if something breaks?*	**Performance** *What is slow or costly?*	**Evaluation** *Does it still work?*
INPUT	an unfinished session offered back	approvals asked of a person	refusals recorded	input ending; interrupts	—	yes to every approval in a test run
CONTEXT	memories; resumed transcripts	untrusted content; poisoned memory; roles and authority	—	compaction when too long	caching the unchanged prefix; trimming results	the instructions under test
MODEL INTERFACE	—	—	tokens, latency, cost per call	retries; a backup model	streaming	the model under test
OUTPUT	—	sandboxes; never-lists; secrets hidden; pinned images	what each action did	time limits on actions	parallel actions; smaller models for side tasks	the tools under test
CONTROL FLOW	the task in progress; a diary of every message	runaway loops; the record kept out of reach	each decision	resume after a crash	—	the loop under test

TABLE II

SIX HARNESSES BY PRIMITIVE, ABRIDGED. EACH CELL IS THE DESIGN DECISION THAT HARNESS MADE WITHIN THE PRIMITIVE; THE FULL TABLE AND THE EVIDENCE FOR EACH CELL ARE PUBLISHED WITH THE PAPER. VERSIONS: CLAUDE CODE DOCUMENTATION AS OF 15 OCTOBER 2026; CODEX CLI F0C5077; OPENCODE 654B79B; PI 5AB7FE7; HERMES AGENT 7BF3766; LANGGRAPH 5EB60AE.

	Claude Code	Codex CLI	opencode	pi	Hermes Agent	LangGraph
INPUT	prompts, stdin, messages queued mid-turn, sub-agent reports	one operation channel per thread; steering queue; mailbox	durable inbox rows, delivered as steer or queue	terminal, print and RPC modes; steer and follow-up queues	one message event from CLI, 25 gateways, HTTP, ACP, cron	writes to a typed state channel, merged by a reducer
CONTEXT	cached prefix (system, tools, `CLAUDE.md`); compaction re-injects memory	Responses item list; world-state sections as diffs; three compaction modes	event-sourced history re-projected per call; compaction at ∼90% or provider-native	projection of a session tree; sectioned prompt diffed into patches	three-tier byte-stable prompt; compaction at 50%, prune then summarize	the state's messages; window-fitting left to a user hook
MODEL INTERFACE	Messages API, streaming; 10 retries; fallback chain	Responses API; WebSocket, falling back to HTTP; 5 retries	in-house request compiled per protocol; retry split by output started	one stream function over ∼15 adapters; overflow to compaction	chat-completions client, pluggable transports; fallback chain	one LangChain call per step; no client of its own
OUTPUT	built-in tools plus MCP; read-only calls concurrent; OS sandbox	one orchestrator: approve, sandbox, run; V8 code mode	tools run concurrently as they stream; code-mode interpreter	plain tool objects, parallel; no core sandbox; QuickJS code mode	core tools plus bridge; sandboxed terminal; approval gate	tool node, concurrent in-process; errors become messages
CONTROL FLOW	loop until no tool calls; six-step permission gate with a classifier model; agent tool to depth 3	task per turn; hooks, Guardian reviewer, then person; sub-agents as sessions	one fibre per session; per-agent step budget; child sessions, depth 1	inner tool loop, outer follow-up loop, session loop; virtual-model routing	phase-verdict loop; stop-gate nudges; flat threaded delegation	a user-written graph run as supersteps; recursion limit

and the enforcement is OUTPUT; by location they are one file.

- **Decisions carried out by editing the context.** opencode enforces a step budget by appending an assistant message and disabling tools; Hermes's stop gates append a synthetic message and go again; LangGraph replaces the model's answer with a canned message at the step budget. Each is a CONTROL FLOW decision whose mechanism is a change to what the model is shown, and we record both.

One finding is a boundary question. Claude Code's server-side permission classifier runs inside the API during the session's own requests, and opencode can delegate compaction to the provider: a function the vocabulary places in the harness runs on the provider's side, and the local harness holds only a verdict or an opaque item. The primitives still apply, with the harness split across two machines. But the boundary of §II is at the model's weights, not at the API, and a deconstruction from client source alone misses the part behind the API. We found no component in the six that satisfied none of the five definitions.

VI. USING THE VOCABULARY

a) Locating a failure: When an agent fails, the primitives give a short list of places to look, in order; each question rules out a place the next assumes.

1) CONTEXT**: was the model shown what it needed?** If the file, the result or the instruction is not in the request that was sent, nothing downstream is at fault.
2) OUTPUT**: did what it asked for happen?** A parser that dropped a call, a sandbox that refused it, a truncated result.
3) CONTROL FLOW**: did the loop stop early or run too long?** A step limit reached early, a hand-back on a response with work still in it, a repeated action.
4) MODEL INTERFACE**: was the response read correctly?** A tool call in the right format but not recognized, a stream cut off and treated as complete.

5) INPUT**: did the right thing come in?** A result delivered to the wrong turn, an interrupt lost.
6) **Only then, the model.**

b) Reporting a harness: If *Agent* = *Harness*(*Model*), a benchmark score belongs to the pair. Holding one model fixed, SWE-agent [8] resolves ten times as many SWE-bench tasks as retrieval-augmented generation, and METR found that tailoring a scaffold to one model more than doubled the length of tasks it could complete [2]. [15] argue that the harness should be disclosed whenever agents are compared, and a disclosure needs a schema. We suggest the five primitives: the input sources; what the context holds and how it is fitted; the provider, decoding settings and retry policy; the tools, their output formats and the execution environment; and the loop's shape and stopping rules. Table II is such a disclosure for six harnesses.

VII. LIMITATIONS

a) What would refute the claim: The position fails if a component of a working harness is found that (i) satisfies none of the five definitions, (ii) is not the answer to one of the six concerns within a primitive, and (iii) is not a nested harness. We invite reports of such a component. Learned components inside the harness and model-generated control flow arose in the six and were placed by the rules; search and decoding loops did not arise and remain untested.

b) Selection and sources: Six harnesses, five of them coding agents; harnesses for browsing, voice or robotics were not examined. Claude Code's deconstruction rests on documentation, which states what the harness does but not always how. Every assignment is pinned to a commit or a date, and several of the six change weekly.

c) Who made the assignments: No second, independent decomposition was made, so the agreement between two people applying the definitions is unmeasured.

d) No experimental test: The paper does not vary one primitive at a time and measure the effect; [16] do this for three decisions, and a study across all five primitives is future work.

VIII. RELATED WORK

The loop that harnesses implement goes back to ReAct [17]. Later patterns live in particular primitives: Reflexion [18] in CONTROL FLOW and CONTEXT, MemGPT [19] in CONTEXT, Toolformer [6] in the model, with the calls still carried out by OUTPUT. CoALA [20] describes what an agent *is*, as a cognitive architecture, where this paper describes what software an agent is *made of*; its memory modules are design decisions in CONTEXT and OUTPUT here, and MODEL INTERFACE and the production concerns have no counterpart there. SWE-agent [8] shows that the agent-computer interface changes performance; in our terms it is a set of decisions in OUTPUT and CONTEXT. Anthropic's distinction between workflows and agents [9] is a property of CONTROL FLOW. [15] argue that harness configuration can account for more variance than the choice of model, and [16] vary planning, the action space and context management, three dimensions that each fall within one primitive; this paper supplies what both assume, a statement of what the harness's parts are.

IX. CONCLUSION

An LLM agent is a model wrapped in a harness, and a harness is five mechanistic primitives: INPUT, CONTEXT, MODEL INTERFACE, OUTPUT and CONTROL FLOW. Harnesses differ in the design decisions made within those primitives, and are made fit to run by the answers they give to six questions asked of each. Six harnesses in wide use fit this description with nothing left over. The vocabulary gives an engineer five places to look before blaming the model, and a benchmark a way to say which harness produced a score.

REFERENCES

[1] Anthropic, "Raising the bar on SWE-bench verified with Claude 3.5 Sonnet," https://www.anthropic.com/research/swe-bench-sonnet, 2025.
[2] METR, "An update on our preliminary evaluations of Claude 3.5 Sonnet and o1," https://metr.org/blog/2025-01-31-update-sonnet-o1-evals/, 2025.
[3] LangChain, "LangGraph," https://github.com/langchain-ai/langgraph, 2026, commit 5eb60ae, 2026-10-14; documentation at https://docs.langchain.com/oss/python/langgraph/overview.
[4] J. Young, "Effective harnesses for long-running agents," https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, Anthropic, 2025.
[5] C. Dovey, "Building agents: From nothing to a model, a harness, and a model trained in its harness," https://github.com/averagejoeslab/building-agents, 2026, book and companion repository, Average Joes Lab.
[6] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, arXiv:2302.04761.
[7] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better LLM agents," in *International Conference on Machine Learning*, 2024, arXiv:2402.01030.
[8] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, arXiv:2405.15793.
[9] E. Schluntz and B. Zhang, "Building effective agents," https://www.anthropic.com/engineering/building-effective-agents, Anthropic, 2024.
[10] Anthropic, "Claude Code documentation," https://code.claude.com/docs/en/how-claude-code-works, 2026, documentation as read on 2026-10-15.
[11] OpenAI, "Codex CLI," https://github.com/openai/codex, 2026, commit f0c5077, 2026-10-15.
[12] Anomaly and contributors, "opencode," https://github.com/anomalyco/opencode, 2026, commit 654b79b, 2026-10-15.
[13] M. Zechner, "pi: a minimal coding agent," https://github.com/badlogic/pi-mono, 2026, commit 5ab7fe7, 2026-10-15.
[14] Nous Research, "Hermes Agent," https://github.com/NousResearch/hermes-agent, 2026, commit 7bf3766, 2026-10-15.
[15] Y. Zhang, J. Wang, Y. Ge, W. Xu, J. Hamm, and C. K. Reddy, "Stop comparing LLM agents without disclosing the harness," *arXiv preprint arXiv:2605.23950*, 2026.
[16] R.-Z. Fan, Z. Zhang, S. Ma, Y. Hu, S. Wang, K. Song, F. Liu, H. Zamani, and X. Wang, "An empirical study of harness design for coding agents," *arXiv preprint arXiv:2609.20804*, 2026.
[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023, arXiv:2210.03629.
[18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, arXiv:2303.11366.
[19] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023.

[20] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents," *Transactions on Machine Learning Research*, 2024, arXiv:2309.02427.